Pretrained Encoders, Transfer Learning

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



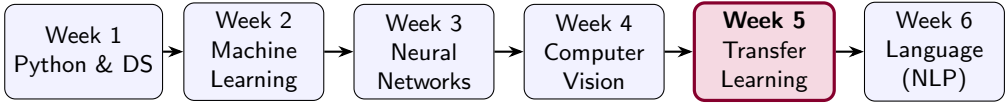
KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 5, Day 29, Lecture

17 October 2026

Table of Contents

1. Big Picture and Motivation
2. Learning Outcomes
3. Pretrained Encoders: What Is in the Box
4. Pretrained Models: The Hugging Face Hub
5. Transfer Learning: Keep the Encoder, Teach a New Head
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References

Where We Are

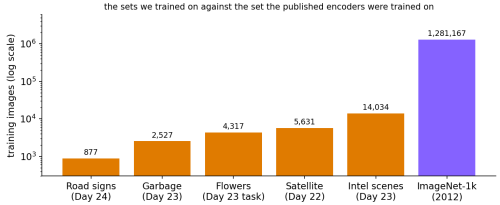


Today: Day 29 of 42

- **Week 4:** images as tensors, convolutions, CNNs trained on Kaggle photos (garbage, satellite scenes, Intel scenes), YOLO and U-Net. Every network we trained ourselves started from *random numbers*; only YOLO started from published weights, and it was the one that worked in 15 epochs.
- **Today:** what published weights are, how to download them from **torchvision** or the **Hugging Face Hub**, and how to reuse the **encoder** for your own classes: **transfer learning**.
- **This week:** Day 30 feature extraction against fine-tuning; Day 31 embeddings, similarity and CLIP; Days 32 and 33 the deep dives; Day 34 the small-data habits; Friday.

How We Did It Before: Every Network Started From Zero

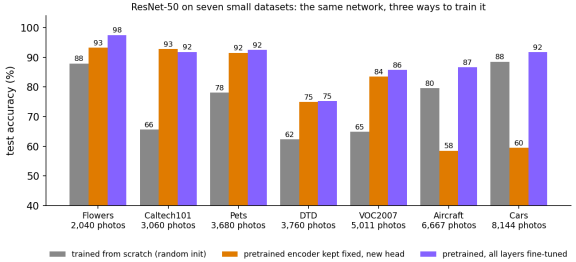
- **Week 4's way:** `SmallCNN()` or `VGGSmall()`; every weight drawn at random, then 10 to 20 epochs on our photos. The network had to learn what an edge is, what a texture is, what a petal is, from our 2,500 to 14,000 photos alone.
- **The networks that won ImageNet** learned the same things from **1,281,167** photos: AlexNet trained for "five to six days on two GPUs" (Krizhevsky et al., 2012); ResNet for weeks of GPU time (He et al., 2016).
- **Day 23's practice scenario:** the flowers from scratch, 4,317 photos, 10 epochs: training accuracy 0.99, validation 0.68. The network memorised the photos instead of learning flowers.



Sources: Russakovsky et al. (2015), Table 2, for ImageNet-1k; the Kaggle dataset pages for the others; `scripts/day29_pretrained_figures.py`.

ImageNet is 300 times our flower set and 1,460 times the road-sign set.

Where It Breaks: A Few Thousand Photos From Scratch



Source: Kornblith, Shlens and Le (2019), Appendix Table I.1, ResNet-50 rows; the from-scratch runs had their own tuned hyperparameters. `scripts/day29_pretrained_figures.py`.

The problem

On 2,000 to 5,000 photos the *same* ResNet-50 reaches 62 to 88% from scratch and 75 to 98% from its ImageNet weights: Caltech-101, 66% against 92%, **26 points** from the same photos and network. Only with 7,000 to 8,000 photos of one kind of object (aircraft, cars) does the gap shrink to a few points.

Today's Idea

In one sentence

Do not start from zero: download a network that already learned to see on 1.28 million photos, keep its **encoder** (the part that turns a photo into features), replace its **head** (the part that names ImageNet's 1,000 classes) with one for *your* classes, and train on your photos. That is **transfer learning**.

- A **pretrained encoder** is the convolutional stages of a network with the weights another team's training produced, published as a file.
- **Where to get one:** `torchvision.models`, PyTorch's own zoo, and the **Hugging Face Hub**, everyone's.

Where you will meet it

IOAI tasks hand you pretrained models as a matter of course: a diffusion pipeline and a DETR detector in 2024, torchvision's ResNet-18 and timm's ViT-Tiny in 2026; the first IOAI 2026 at-home task was, in short, "take a deployed classifier, reuse its encoder, teach it 13 new classes in 10 minutes of GPU". Kaggle's image competitions are won from pretrained weights. Today's labs load ResNet-18 from torchvision and from the Hub, then teach it five kinds of flower.

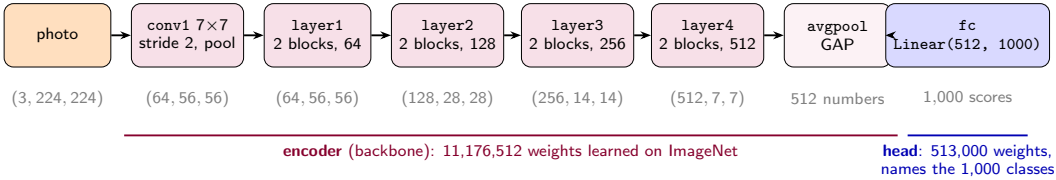
⇒ Learning Outcomes

By the end of this session, you will be able to:

- **Explain** what a pretrained encoder is: the two parts of a classifier, what the encoder's weights learned on ImageNet, and what its 512-number output is.
- **Load** a published model from `torchvision` with its weights and its preprocessing, and run it on a photo the standard way.
- **Find** a model on the Hugging Face Hub, read its model card, and load it with `transformers` (the pipeline and the explicit way).
- **Explain** transfer learning, why it works, and what the published numbers say it buys on a few thousand photos.
- **Replace** the head of a pretrained ResNet-18 for your classes, freeze the encoder, and train the head with Week 3's loop and Week 4's data pipeline.
- **Recognise** a transfer-learning task in an Olympiad statement and pick the encoder and the training strategy.

Pretrained Encoders: **What Is in the Box**

Every Classifier Has Two Parts: Encoder and Head



Definition

The **encoder** is everything up to the pooled vector: it turns a photo into a fixed list of numbers (512 for ResNet-18, 2,048 for ResNet-50). The **head** is the last `Linear`: one score per class from those numbers. Day 22 drew the same line through `SmallCNN`: `features` and `classifier`.

- The encoder holds 96% of the weights and all of the "seeing". The head is one matrix that names things, and the names are ImageNet's. Yours are different: that is the whole story of today.

Pretrained on ImageNet-1k: The Weights Are a Published File

Definition

A **pretrained model** is an **architecture** (code: `resnet18()`) plus a **weights file**: the numbers produced by training it on **ImageNet-1k**, Day 23's 1,281,167 photos and 1,000 classes, about 90 epochs, days of GPU time. Load the file into the architecture and you hold that training without repeating it.

Tensor in `resnet18-f37072fd.pth`	**Shape**	**Numbers**
`conv1.weight`	(64, 3, 7, 7)	9,408
`layer1.0.conv1.weight`	(64, 64, 3, 3)	36,864
`layer4.1.conv2.weight`	(512, 512, 3, 3)	2,359,296
`fc.weight`	(1000, 512)	512,000
`fc.bias`	(1000,)	1,000
122 tensors in all		11,689,512

Read from the published file with `model.state_dict()`;

`scripts/day29_pretrained_figures.py`.

- 11.7 million numbers, 44.7 MB, downloaded once: Day 16's `torch.save(model.state_dict())`, run by the PyTorch team after their ImageNet training.
- `conv1.weight`: 64 kernels of 7×7 over 3 channels, Day 22's kernels, learned instead of hand-written.
- The 1,000 classes include `daisy` but no `tulip` or `glacier`. Offline contests hand you the file.

PyTorch's Own Zoo: `torchvision.models`

Model	Year	Parameters	Weights file	ImageNet top-1	When people pick it
ResNet-18	2015	11.7 M	44.7 MB	69.8%	the default first try: fast, forgiving
ResNet-50 (V2 weights)	2015	25.6 M	97.8 MB	80.9%	more accuracy for 4 times the compute
MobileNetV3-Large	2019	5.5 M	21.1 MB	75.3%	a CPU, a phone, a tight clock
EfficientNet-B0	2019	5.3 M	20.5 MB	77.7%	small and accurate; wants 224 pixels
EfficientNet-V2-S	2021	21.5 M	82.7 MB	84.2%	the accurate CNN when time allows
ConvNeXt-Tiny	2022	28.6 M	109 MB	82.5%	a modern CNN, strong for fine-tuning
ViT-B/16	2021	86.6 M	330 MB	81.1%	the transformer; Day 31 and Week 6

Source: the torchvision model table, read from each `Weights.DEFAULT.meta` (`num_params`, `_file_size`, `acc@1` on ImageNet-1k val, 224 pixels). 80 classification models in all.

- Every model comes as a **function** (`resnet18`) and a **weights enum** (`ResNet18_Weights.IMAGENET1K_V1`); `.DEFAULT` is the best available. The enum carries the facts above and the preprocessing the weights expect.
- Read the table like a menu: **parameters** set the time per epoch and the memory; **top-1** says how well the encoder sees. Common practice: start with ResNet-18, move up only when the validation score and the clock both allow.

In Code: Load, Preprocess, Predict, the Standard Way

```python
from torchvision.models import resnet18, ResNet18_Weights

weights = ResNet18_Weights.DEFAULT              # IMAGENET1K_V1: 69.8% top-1
model = resnet18(weights=weights)               # downloads 44.7 MB once, loads it
model.eval()                                    # BatchNorm in inference mode (Day 17)
preprocess = weights.transforms()               # resize 256, centre crop 224, normalise

img = Image.open("sunflower.jpg").convert("RGB")
x = preprocess(img).unsqueeze(0)                # (1, 3, 224, 224): a batch of one
with torch.no_grad():                           # no gradients: faster, less memory
    probs = model(x).softmax(dim=1)[0]          # 1,000 probabilities
top = probs.topk(3)
names = weights.meta["categories"]              # the 1,000 class names
print([(names[i], round(p.item(), 2)) for p, i in zip(top.values, top.indices)])
```

- Output on the sunflower photo: `[('daisy', 0.93), ('bee', 0.02), ('buckeye', 0.02)]`; `weights.meta["categories"]` holds the 1,000 names in output order. Three habits of every prediction cell: `model.eval()`, `torch.no_grad()`, `unsqueeze(0)`.
- Older spellings you will meet: `models.resnet18(pretrained=True)` (deprecated since 2022, still everywhere) and `resnet18(weights="IMAGENET1K_V1")`, the string form of the PyTorch tutorial. Same weights.

The Preprocessing Contract



Source: a photo from Kaggle's *Flowers Recognition* set through `ResNet18_Weights.DEFAULT.transforms()`, step by step; `scripts/day29_pretrained_figures.py`.

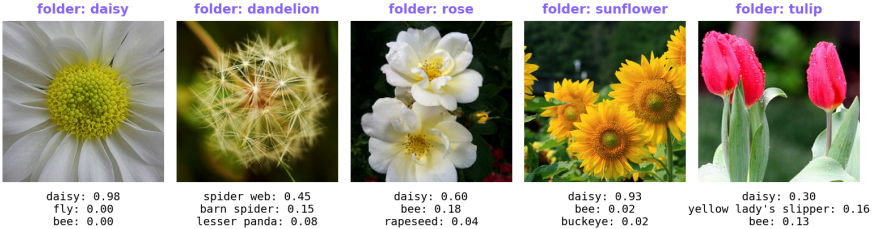
The contract

What the weights were trained on: `Resize(256)`, `CenterCrop(224)`, `ToTensor()`, then `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`: ImageNet's channel means and standard deviations.

- **One pixel through it.** The centre pixel is stored as $(232, 182, 20)$. `ToTensor` divides by 255: $(0.910, 0.714, 0.078)$. `Normalize`: red $(0.910 - 0.485)/0.229 = 1.855$, blue $(0.078 - 0.406)/0.225 = -1.456$.
- **Where it bites:** feed pixels 0 to 255, or 0 to 1 without the normalisation, and the weights still run and print probabilities; the answers are simply wrong. For training, keep the same `Normalize`, augmentation in front of it.

What the Pretrained Classifier Says About Your Photos

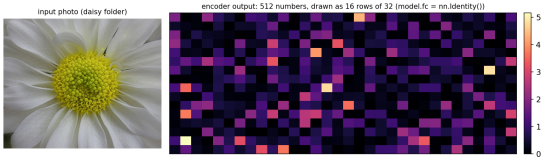
ResNet-18 with its published ImageNet weights: top-3 of its 1,000 labels for one photo per folder



Source: ResNet-18 with its published ImageNet weights, applied once to the first photo of each folder of Kaggle's *Flowers Recognition* set; `scripts/day29_pretrained_figures.py`.

- ResNet-18 *knows* `daisy`, one of its 1,000 classes: 0.98. It has no class for dandelion, rose, sunflower or tulip, so it answers with the nearest thing it knows: `daisy` for the rose and the sunflower, `spider web` for the dandelion's seed head.
- The encoder saw the flowers well: the 512 numbers for the tulip differ from the daisy's. Only the head has no row for `tulip`. Keep the encoder, change the head: the next section.

The Encoder's Output: 512 Numbers per Photo



Source: the published ResNet-18 encoder on the daisy photo of the previous frame; `scripts/day29_pretrained_figures.py`.

```
encoder = resnet18(weights=weights)
encoder.fc = nn.Identity()    # drop fc
features = encoder(x)         # (1, 512)
```

Definition

The vector after the global average pool is the photo's **feature vector**: one number per channel of `layer4`, averaged over its 7×7 map. People also say *embedding*; Day 31 lives there.

- For the daisy: 512 numbers from 0 to 5.15, none negative (they leave a ReLU), 12 exactly zero. A second daisy gives a similar pattern, a tulip a different one. The head is one `Linear` on this vector, so a head for *your* classes is `Linear(512, k)` on the same vector.

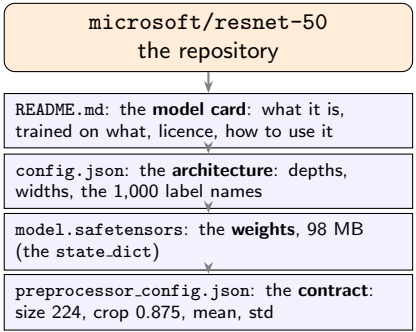
Pretrained Models: **The Hugging Face Hub**

What the Hub Is

Definition

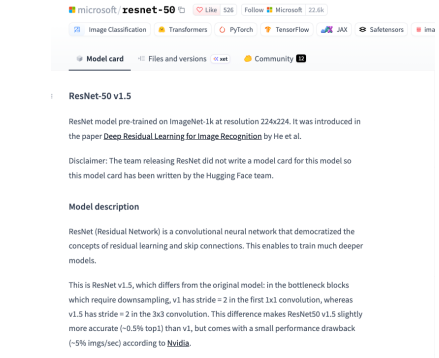
huggingface.co is a GitHub for models. Every model is a Git repository with a page (the **model card**), the weights file, a config and the preprocessing settings. It hosts over 2 million models and 1.5 million datasets.

- A model is named `owner/name`: `microsoft/resnet-50`, `timm/resnet18.a1_in1k`.
- The `transformers` library downloads the files and builds the model: the Hub's counterpart of torchvision's `weights=`.
- torchvision has 80 classification models; the Hub lists 29,801 for image classification alone.



The four files of `huggingface.co/microsoft/resnet-50`, as downloaded by `transformers` (October 2026).

A Model Page: Reading the Card



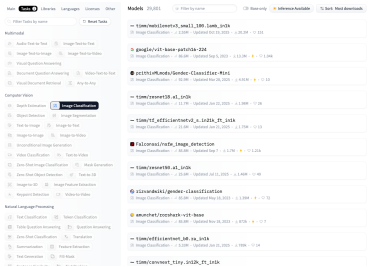
Source: `huggingface.co/microsoft/resnet-50`, screenshot of 18 October 2026.

What to read, top to bottom:

1. The **task** tag: Image Classification.
2. The **libraries**: Transformers, PyTorch.
3. The **training data**, `imagenet-1k`, and the **paper**, `arxiv:1512.03385`: what the encoder learned, where it is described.
4. The **licence**, `apache-2.0`: usable in a contest and in a product.
5. "Pre-trained on ImageNet-1k at resolution 224x224": the preprocessing contract, on the page.
6. **Files and versions**: the four files; **downloads last month**: 294,656.

Rule of thumb: a card naming data, paper and licence is a model to build on; no card and ten downloads is not.

Finding a Model: Filter by Task, Sort by Downloads



Source: `huggingface.co/models`, task Image Classification, sorted by downloads, screenshot of 18 October 2026.

- Filter by **task** (left), sort by **downloads**: `timm/mobilenetv3_small_100`, `google/vit-base-patch16-224` (86.6 M), `timm/resnet18.a1_in1k` (11.7 M), `timm/efficientnet_b0`: torchvision's families again.
- Each entry prints its **parameter count**: 11.7 M against 86.6 M tells you the time per epoch before downloading. `in1k` in a timm name means ImageNet-1k weights.

In Code: The Hub in Three Lines, and the Explicit Way

```python
from transformers import pipeline
clf = pipeline("image-classification", model="microsoft/resnet-50")
print(clf(img)[:2])      # [{'label': 'daisy', 'score': 0.993}, {'label': 'vase', ...}]
```

```python
from transformers import AutoImageProcessor, AutoModelForImageClassification
processor = AutoImageProcessor.from_pretrained("microsoft/resnet-50")
model = AutoModelForImageClassification.from_pretrained("microsoft/resnet-50")
inputs = processor(images=img, return_tensors="pt")      # pixel_values: (1, 3, 224, 224)
with torch.no_grad():
    logits = model(**inputs).logits                      # (1, 1000)
print(model.config.id2label[logits.argmax(-1).item()])  # 'daisy'
```

- The **pipeline** bundles preprocessing, model and labels into one callable: for looking, not for training. The **explicit way** is torchvision's three steps under other names: `AutoImageProcessor` is `weights.transforms()`, `from_pretrained` is `resnet18(weights=...)`.
- The daisy: 0.993 from the Hub's ResNet-50 against 0.984 from torchvision's ResNet-18; same data, bigger model, surer. `from_pretrained` also takes a *folder*: offline contests hand models over as folders (IOAI 2026).

Three Doors to the Same Weights: Which to Use When

	`torchvision.models`	**Hugging Face** `transformers`	`timm`
who	the PyTorch team	Hugging Face and everyone	Ross Wightman; weights on the Hub
vision models	80 classification models	tens of thousands	about 1,800 with pretrained weights
load	`resnet18(weights=ResNet18_Weights.DEFAULT)`	`AutoModelForImageClassification.from_pretrained(id)`	`timm.create_model("resnet18", pretrained=True)`
preprocessing	`weights.transforms()`	`AutoImageProcessor`	`timm.data.create_transform`
new head for k classes	`model.fc = nn.Linear(512, k)`	`num_labels=k, ignore_mismatched_sizes=True`	`num_classes=k`
best for	the default in a competition notebook; CNNs	models beyond torchvision: ViT, CLIP, text, audio (Days 31, 36 to 38)	the widest menu of CNNs and ViTs; the Kaggle favourite

- The same ResNet-18 weights sit behind all three doors; the door decides the spelling. Learn one well (torchvision today) and read the other two.
- IOAI 2026's *Double Agent Dilemma* handed contestants `torchvision.models.resnet18` and timm's `vit_tiny_patch16_224` in one task (Individual Contest, Task 4). Today's labs train with torchvision and look with the Hub; Day 31 and Week 6 use `transformers` for CLIP and BERT.

Transfer Learning: **Keep the Encoder, Teach a New Head**

Definition

Transfer learning

Train a network on a task where data is plentiful (ImageNet: 1.28 million photos, 1,000 classes), then **reuse what it learned** on a different task where data is scarce (your 4,317 flowers, 5 classes). The reused part is the **pretrained encoder**; the new task gets a **new head** and a little training.

Analogy

A student who already reads Arabic learns to read a chemistry textbook far faster than one who starts with the alphabet: the letters, the words and the reading habits transfer; only the chemistry is new. The encoder already knows edges, textures, petals and fur; only "which of your five classes" is new.

	From scratch (Week 4)	**Transfer learning (today)**
starting weights	random	the published ImageNet weights
what must be learned	everything, edges to classes	the head, and a little tuning
photos needed	thousands per class	tens to hundreds per class
epochs	20 to 100	3 to 10
ResNet-50 on 2,040 flower photos	87.9%	93.2% (head only), 97.5% (fine-tuned)

Last row: Kornblith, Shlens and Le (2019), Appendix Table I.1, Oxford 102 Flowers.

Why It Works: Early Layers Are General, Late Layers Are Specific



Source: Lee, Grosse, Ranganath and Ng (2009), Fig. 3: what units of layers 1, 2 and 3 of a convolutional network trained on faces respond to. Reused from Day 23.

- **Day 23's picture:** layer 1 responds to edges, layer 2 to parts, layer 3 to whole objects. Edges look the same in any photo, so the first layers fit any photo; the deepest lean towards ImageNet's objects; the head is ImageNet only.
- **Measured:** Yosinski et al. (2014) transplanted layers between networks: first-layer features "appear not to be specific to a particular dataset or task"; transferability falls as the new task moves away from the old; transferred features give "a boost to generalization" that "lingers even after fine-tuning".

The Evidence, With Numbers

ResNet-50 on	photos	scratch	head only	fine-tuned
Oxford Flowers	2,040	87.9	93.2	97.5
Caltech-101	3,060	65.6	92.7	91.8
Oxford Pets	3,680	78.1	91.5	92.5
DTD (textures)	3,760	62.3	74.9	75.2
PASCAL VOC 2007	5,011	64.9	83.5	85.7

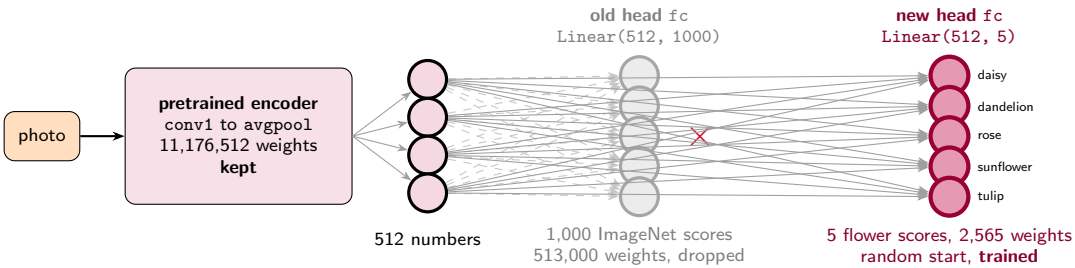
Test accuracy in %; Kornblith, Shlens and Le (2019), Appendix Table I.1. "Head only": the encoder kept fixed, a logistic-regression head trained on its features.

ResNet-18, ants against bees	val. accuracy	time
encoder frozen, head trained	94.8%	28 s
all layers fine-tuned	94.8%	37 s

About 120 training photos per class, 75 validation each, 25 epochs; the PyTorch transfer-learning tutorial's printed results.

- With a few thousand photos the pretrained start adds **10 to 27 points**. The fixed encoder already captures most of the gain; fine-tuning adds a few points and costs more time.
- **240 photos and half a minute** reach 95%: the regime of a competition day, and the regime where training from scratch gives nothing.
- These are other people's measurements on public sets. Your own number comes from your validation split, as always; Lab 2 shows what the flowers give.

The Mechanism: Swap the Head



1. **Load** the pretrained model: `resnet18(weights=ResNet18_Weights.DEFAULT)`.
2. **Decide what moves:** freeze the encoder (`requires_grad = False`) or leave it trainable.
3. **Replace the head:** `model.fc = nn.Linear(512, 5)`, a fresh `Linear` with random weights.
4. **Train** with Week 3's loop on your photos (the weights' preprocessing), 3 to 10 epochs; validate, keep the best epoch, predict.

With Numbers: What Is Learned, What Is Kept

Part	Layers	Parameters	In transfer learning
encoder	`conv1`, `bn1`, `layer1` to `layer4`	11,176,512	kept; learned on 1.28 million photos
old head	`fc = Linear(512, 1000)`	513,000	thrown away
new head	`fc = Linear(512, 5)`	2,565	learned on your 4,317 photos

The new head, in math

With the encoder's vector $h \in \mathbb{R}^{512}$, weights $W \in \mathbb{R}^{5\times 512}$ and bias $b \in \mathbb{R}^5$:

$$z = Wh + b, \qquad p = \mathrm{softmax}(z), \qquad L = -\log p_{\text{true class}}$$

$512 \cdot 5 + 5 = 2{,}565$ numbers to learn; probabilities and loss as on Day 16.

- With a frozen encoder, **2,565 of 11,179,077** parameters move: 0.02%. The rest were fitted by 1.28 million photos and stay.
- **Photos per learned parameter:** $4{,}317/2{,}565 = 1.7$ today, against $4{,}317/288{,}000 = 0.015$ for Day 23's `VGGSmall` from scratch. That ratio is why one memorises and the other does not.
- ResNet-50: $h \in \mathbb{R}^{2048}$, a new head of $2{,}048 \cdot 5 + 5 = 10{,}245$ numbers.

In Code: The Standard Transfer-Learning Snippet

```python
from torchvision.models import resnet18, ResNet18_Weights

model = resnet18(weights=ResNet18_Weights.DEFAULT)
for param in model.parameters():
    param.requires_grad = False                    # freeze everything from ImageNet
model.fc = nn.Linear(model.fc.in_features, 5)      # new head, 512 -> 5, trainable
model = model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)   # only the head moves
```

- These lines are in every Kaggle transfer-learning notebook and in the PyTorch tutorial. `model.fc.in_features` reads the 512 from the model instead of hard-coding it.
- The head has a different name in each family: `fc` for ResNet, `classifier[-1]` for MobileNet, EfficientNet and VGG, `heads.head` for ViT. `print(model)` shows the last line; that is the one to replace.
- Freezing has two halves: `requires_grad = False` stops the gradients; giving the optimizer *only* `model.fc.parameters()` stops the updates. The new `Linear` is created after the loop, so it is trainable. To fine-tune everything, drop the loop and pass `model.parameters()` with `lr=1e-4`: tomorrow.

The Same Data Pipeline and the Same Loop

```
normalize = transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
train_tf = transforms.Compose([transforms.RandomResizedCrop(224),
                               transforms.RandomHorizontalFlip(), transforms.ToTensor(),
                                   normalize])
val_tf = transforms.Compose([transforms.Resize(256), transforms.CenterCrop(224),
                             transforms.ToTensor(), normalize])
train_ds = datasets.ImageFolder("flowers/train", transform=train_tf)   # labels from
    folders
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True, num_workers=2)

for epoch in range(5):
    model.train()
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        loss = criterion(model(xb), yb)                    # 1. predict  2. loss
        optimizer.zero_grad(); loss.backward(); optimizer.step()   # 3. gradient  4.
            update
    model.eval()                                           # validation pass, keep the
        best epoch
```

- Not one line of Week 3's loop changed; the pipeline is Day 23's with the ImageNet `Normalize` and 224 pixels. With a frozen encoder a step is one forward pass and a 2,565-number update: an epoch of 3,453 photos takes under a minute on Colab's T4.

Two Ways to Train It: Named Today, Taken Apart Tomorrow

	Feature extraction (encoder frozen)	**Fine-tuning** (all layers trainable)
what moves	the new head only	every weight, starting from the pretrained values
learning rate	10^{-3} for the head	10^{-4} or smaller: small steps keep the knowledge
epochs	3 to 10	3 to 10, often fewer
time per epoch	short: no gradients through the encoder	2 to 3 times longer
photos per class	works from tens	wants hundreds
Oxford Flowers, ResNet-50	93.2%	97.5%
risk	the encoder cannot adapt to unusual photos	with few photos or a big step it forgets what it knew

Sources: Kornblith et al. (2019); the PyTorch transfer-learning tutorial; Yosinski et al. (2014). The IOAI 2026 at-home task warned that "naive training ... will torch the old classes within a few hundred steps".

- **Common practice:** start frozen; read the validation accuracy; if time remains, unfreeze the last stage (`layer4`) with a small learning rate and train a few more epochs. Freezing some layers and not others is **layer freezing**: Day 30 and Day 32 do it step by step, with the numbers.
- Today's Lab 2 does the first line of that practice on the flowers and reads what it gives.

Transfer Learning: **Olympiad Practice**

Practice Task: Teach an Old Classifier New Classes

Task

A deployed classifier, a pretrained encoder (86 million parameters, 768-number feature vectors) with a head for **16 classes**, must learn **13 new classes** for two new customers. You get the checkpoint, 24 to 60 labelled clips per new class, one GPU and **10 minutes of training**. Rules: start from the checkpoint and reuse its encoder; one model, all 29 classes, one forward pass; no training from scratch, no other pretrained model. Score: the mean of the accuracy on the old classes and on the new ones.

- **Question 1:** which part of the checkpoint do you keep and which do you replace? How many parameters does the new head have?
- **Question 2:** before training, you run the 16-class model on the new clips. What answers do you expect, and what do they tell you? (Think of the flowers frame.)
- **Question 3:** with 10 minutes and 24 to 60 clips per class, which training way do you start with, and which number do you watch so the old customers keep their accuracy?

Source: IOAI 2026, At-Home Round, Task 1, "Operation Night Watch" (`github.com/IOAI-official/IOAI-2026`). The original is audio (an Audio Spectrogram Transformer); its mechanics are today's. Its second half, fighting forgetting with replay and distillation, is beyond this course.

Solution Idea and Code

Recognise: transfer learning under a budget: a pretrained encoder to keep, a head that must grow from 16 rows to 29.

```
old = model.classifier                                  # Linear(768, 16): the deployed head
new = nn.Linear(old.in_features, 29)                    # 768 * 29 + 29 = 22,301 parameters
with torch.no_grad():
    new.weight[:16] = old.weight; new.bias[:16] = old.bias   # keep the 16 old rows
model.classifier = new
for param in model.encoder.parameters():
    param.requires_grad = False                         # 86 million weights kept
optimizer = torch.optim.Adam(model.classifier.parameters(), lr=1e-3)
```

1. Keep the encoder: 86 million weights from millions of clips cannot be re-learned in 10 minutes. New head: a 29-row `Linear`, $768 \cdot 29 + 29 = 22{,}301$ parameters, 0.03% of the model; the 16 old rows are copied in.
2. The 16-class model *must* answer with one of its classes: a chainsaw lands on "engine" or "thunderstorm", as the tulip landed on `daisy`. The encoder hears the difference; the head has no row.
3. Start frozen: a 22,301-parameter head trains in seconds. Mix old clips into every batch so the old rows keep training (half the score is theirs); watch per-class validation accuracy for *both* groups; unfreeze the last blocks at 10^{-5} only if time remains.

Transfer Learning: **Limitations & What Comes Next**

Limitations

- **Pretrained is not free of assumptions.** The encoder expects photos like ImageNet's: everyday objects, three channels, 224 pixels, ImageNet's normalisation. X-rays, satellite tiles and spectrograms are further away; transfer helps less, more layers need retraining (Day 30).
- **Enough data and time close the gap.** He, Girshick and Dollár (2019) trained detectors from random weights on COCO's 118,000 images and, with a longer schedule, matched the pretrained ones: pre-training "speeds up convergence early in training" but "does not necessarily ... improve final target task accuracy". Millions of images and days of GPU: scratch is an option; thousands and minutes: no.
- **The weights carry their data's biases and their licence.** A model card tells you both; an unknown repository's file tells you neither.
- **Size costs.** ResNet-50 needs about four times ResNet-18's compute; in a 10-minute budget the smallest encoder that reaches the accuracy wins.

The common mistake

Feeding the encoder photos it was never told about: pixels 0 to 255 instead of normalised, 64-pixel images instead of 224, one grey channel instead of three. Nothing crashes; the accuracy is low. Use `weights.transforms()` or copy its four steps, then print `xb.shape`, `xb.mean()`, `xb.std()` of one batch: $(32, 3, 224, 224)$, a mean near 0, a standard deviation near 1.

What Comes Next

- **Tomorrow (Day 30):** feature extraction against fine-tuning with the numbers, layer freezing, one learning rate per part of the network, and the small-data regime.
- **Day 31:** the encoder's 512 numbers as **embeddings**: similarity between photos, nearest neighbours, and CLIP, which puts photos and sentences in the same space.
- **Days 32 and 33, self-study:** freezing step by step; embedding models, linear probing, CLIP zero-shot classification and retrieval.
- **Day 34, Olympiad practice:** the small-data fine-tuning habits that win points. **Friday:** the Week 5 competition.
- **Today's labs:** Lab 1 loads ResNet-18 from torchvision and from the Hub and reads what each says about Kaggle flower photos; Lab 2 teaches ResNet-18 the five flowers with a frozen encoder, end to end, and writes the submission.

Transfer Learning: **Summary**

Summary

- **Two parts:** the **encoder** (`conv1` to `avgpool`, 11,176,512 weights in ResNet-18) turns a photo into 512 numbers; the **head** (`fc`, 513,000 weights) names ImageNet's 1,000 classes. **Pretrained**: the weights file of a run on 1,281,167 photos is published, 44.7 MB, loaded in one line.
- **torchvision:** `resnet18(weights=ResNet18_Weights.DEFAULT)`; `weights.transforms()` is the contract (resize 256, crop 224, ImageNet mean and std); `model.eval()` and `torch.no_grad()` to predict. In the zoo's table, parameters set the time and top-1 the quality.
- **The Hub:** a repository per model with a card, a config, the weights and the preprocessing; `pipeline(...)` to look, `AutoImageProcessor` and `AutoModelForImageClassification` to work; `timm` is the third door to the same weights.
- **Transfer learning:** keep the encoder, replace the head, train on your photos. Early layers are general, late layers specific (Yosinski et al., 2014); on 2,000 to 5,000 photos it adds 10 to 27 points over scratch (Kornblith et al., 2019); 240 photos reach 95% in half a minute.
- **In code:** `requires_grad = False`, `model.fc = nn.Linear(model.fc.in_features, k)`, an optimizer on `model.fc.parameters()`, Week 3's loop and Day 23's pipeline with the ImageNet `Normalize`: 2,565 numbers learned, 11,176,512 kept.
- **Two ways:** feature extraction (frozen, 10^{-3}, fast, few photos) and fine-tuning (everything, 10^{-4}, slower, more photos, a few points more). Start frozen; Day 30 takes both apart.

Transfer Learning: **References & Resources**

References & Resources

[1] Russakovsky, O. et al. (2015). ImageNet large scale visual recognition challenge. *IJCV* 115. The 1,281,167 training photos and 1,000 classes.

[2] Krizhevsky, A., Sutskever, I. and Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS* 25. "Five to six days on two GTX 580 GPUs".

[3] He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *CVPR*.

[4] Lee, H., Grosse, R., Ranganath, R. and Ng, A. (2009). Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. *ICML*. Fig. 3 reused.

[5] Yosinski, J., Clune, J., Bengio, Y. and Lipson, H. (2014). How transferable are features in deep neural networks? *NeurIPS* 27. General first layers, specific last layers.

[6] Kornblith, S., Shlens, J. and Le, Q. (2019). Do better ImageNet models transfer better? *CVPR*. Appendix Table I.1, ResNet-50 rows: from scratch, logistic regression on fixed features, fine-tuned.

[7] He, K., Girshick, R. and Dollár, P. (2019). Rethinking ImageNet pre-training. *ICCV*.

References & Resources (cont.)

[8] PyTorch tutorials, *Transfer learning for computer vision*: ants against bees, about 120 training photos per class; best validation accuracy 0.9477 in 37 s and 28 s.
https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial.html

[9] torchvision documentation: `torchvision.models`, the weights enums and their `meta`, `weights.transforms()`.
https://pytorch.org/vision/stable/models.html

[10] Hugging Face Hub documentation ("over 2M models, 1.5M datasets"), model cards, the `transformers` pipeline and Auto classes; `microsoft/resnet-50` and the image-classification listing, screenshots of 18 October 2026.
https://huggingface.co/docs/hub Wightman, R. (2019). PyTorch Image Models (`timm`).
https://github.com/huggingface/pytorch-image-models

[11] IOAI 2026 (Astana): At-Home Round, Task 1, "Operation Night Watch" (the practice task); Individual Contest, Task 4, "Double Agent Dilemma" (torchvision's `resnet18`, timm's `vit_tiny_patch16_224`), Task 3, "Potato".
https://github.com/IOAI-official/IOAI-2026 IOAI 2024 (Burgas): "The Madarian Cow Mystery": a diffusion pipeline and `facebook/detr-resnet-101` via `from_pretrained`. https://github.com/IOAI-official/IOAI-2024

[12] Lab dataset from Kaggle through `kagglehub`: *Flowers Recognition* (`alxmamaev/flowers-recognition`, 4,317 photos in five folders), also behind the photos in this deck. Figures without another source: `slides-builder/scripts/day29_pretrained_figures.py`; no model was trained for this deck, the only model runs are the published ResNet-18 weights applied once to these photos, and every comparison number is quoted from the sources above.